Materials Learning Algorithms Package

Attila Cangi[a,b,*], Lenz Fiedler[a,b], Andrew Rohskopf[c], Bartosz Brzoza[a,b], Daniel Kotik[a,1], Steve Schmerler[a], Dayton J. Vogel[c], Normand Modine[c], Aidan P. Thompson[c], Sivasankaran Rajamanickam[c]

[a] *Helmholtz-Zentrum Dresden-Rossendorf, Dresden, 01328, Germany*
[b] *Center for Advanced Systems Understanding, G*
ΩΩimmediateörlitz, 02826, Germany
[c]Sandia National Laboratories, Albuquerque, 87123, NM, United States

Abstract

Write me...

Keywords: Machine Learning, Density Functional Theory

PROGRAM SUMMARY/NEW VERSION PROGRAM SUMMARY

Program Title: Materials Learning Algorithms (MALA)
CPC Library link to program files: (to be added by Technical Editor)
Developer's repository link: `https://github.com/mala-project/mala`
Code Ocean capsule: (to be added by Technical Editor)
Licensing provisions(please choose one): BSD 3-clause
Programming language: Python, Fortran
Supplementary material: None
Nature of problem(approx. 50-250 words): Advanced materials research requires quantum-accurate materials properties at experimentally relevant conditions. Traditional electronic structure methods, most notably density functional theory (DFT), cannot satisfy this demand due to their adverse scaling behavior. Such simulations typically cannot treat more than a few thousand atoms, while computational cost simultaneously scales cubically with growing temperature. Large-scale investigations thus have to resort to approximations, classical models or machine learning (ML) approaches. Current ML-DFT approaches are limited in applicability, as they only recover a portion of the predictive capabilities of electronic structure calculations.

*a.cangi@hzdr.de

Solution method(approx. 50-250 words): The MALA code implements an efficient and scalable ML workflow, substituting the need for DFT calculations in extended simulations. Based on a universal representation of the electronic structure, models can recover a range of fundamental observables, from which materials properties may be computed. MALA delivers a unified library for training data sampling, model training and scalable inference by implementing interfaces to state-of-the art simulation codes such as Quantum ESPRESSO [1] and LAMMPS [2].
Additional comments including restrictions and unusual features (approx. 50-250 words): None

References

[1] P. Giannozzi et al., QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials, J. Phys.: Condens. Matter 21, 95502 (2009).

[2] A. P. Thompson et al., LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Comp. Phys. Comm. 271, 108171 (2022).

1. Introduction

Who? Attila

- Introduce concepts of electronic structure theory, materials science, and machine learning.
- Motivate why it is useful to combine machine learning with electronic structure theory
- List some relevant prior works. We can rely on our review article. But also remember to look into the newer literature from the past three years.
- *Figure: Maybe something from the review article?*

2. Theoretical background

Who? Attila

- Introduce the electronic structure problem starting from the non-relativistic Schrödinger equation for the coupled electron-ion problem.
- Reduce it to the electronic Schrödinger equation within the Born-Oppenheimer approximation.
- Introduce density functional theory and the Kohn-Sham equations on a formal level and describe why this is a useful method to solve the electronic structure problem.
- Formulate the DFT problem in a suitable fashion for machine learning starting from the usual challenges and ending up with the representation in terms of the LDOS.
- Provide the mathematical basis of the MALA machine learning model that is based on learning the LDOS.

3. Numerical implementation

3.1. Code structure

Realization of a generally applicable ML-DFT workflow is challenging, since it involves a variety of tasks, such as data sampling and preparation or model training, testing and application. Furthermore, the resulting computer code needs to address a diverse range of requirements from potential users. While material scientist may primarily be interested in applying pretrained models, researchers preparing said models need fine-grained control over training routines.

To this end, the MALA package has been developed in a modular fashion. Based on a central collection of control parameters, users construct individual workflows tailored to their needs. Such workflows largely consist of exchangeable building blocks delegating the principal computational tasks to external or internal, python-based libraries. Fig. 1 details which tasks such a workflow may encompass. Additionally, the usage of internal and external libraries is given.

It can be seen that the MALA code functions as a connector between many different computational tasks, which are partially realized in python

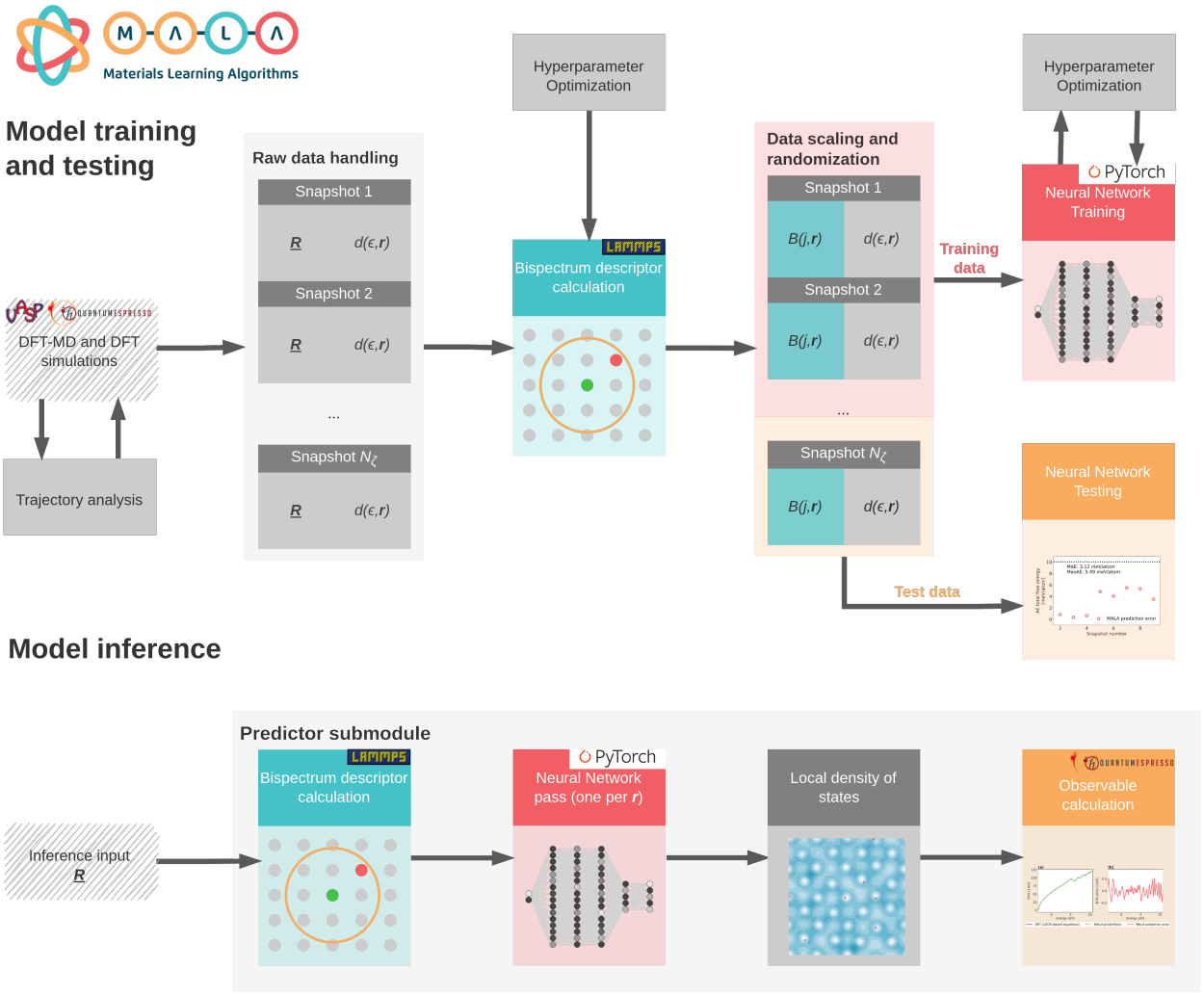

Figure 1: Overview of MALA workflow. Both model training procedure and inference are shown. For the latter, relevant routines are bundled in a separate class within the MALA package to facilitate access. Gray boxes denote pure python-based routines, while blue, red and orange boxes denote that the majority of the computational workload is offloaded to the external libraries LAMMPS, Pytorch and Quantum ESPRESSO, respectively

directly or via the LAMMPS, Pytorch and Quantum ESPRESSO libraries. Task related to obtaining suitable training data, i.e., trajectory analysis and bispectrum calculation, are discussed in Sec. 3.2, while data scaling, randomization and management is discussed in Sec. 3.4. Information on model training, including hyperparameter optimization, is given in Sec. 4.2, while observable calculation, which is part of both

Who? Lenz

- Explain general layout of code
- Outline depdendencies to QE and LAMMPS
- *Figure: Workflow overview of code*

3.2. Data sampling

Sampling training data is a crucial task of any ML-based endeavor, as model accuracy is always limited by the quality of the training data. Curating data sets within an LDOS-based ML-DFT workflow comes with three challenges. Firstly, one has to identify representative ionic configurations on which to perform electronic structure simulations. Secondly, the ionic structure of these configurations has to be adequately represented on a suitable numerical grid. Finally, one has to properly extract the LDOS from DFT simulations.

The first of these tasks is achieved via DFT-MD simulations. By coupling ionic dynamics and electronic structure calculations, materials can be simulated at relevant conditions. To extract sets of suitable configurations, an equilibration analysis as outlined in Ref. [**?**] can be performed via the MALA code. To this end, one computes radial distribution functions (RDF) of an ionic configuration as

$$g(r) = \frac{\Xi(r)}{\rho_\mathrm{i} N_\mathrm{i} V_\mathrm{shell}} \,, \tag{1}$$

with the ionic density ρ_i and average number of ions $\Xi(r)$ in a shell of volume V_shell and radius $r + dr$. Then, based on a portion at the end of an DFT-MD trajectory which can reasonably assumed to be equilibrated, a reasonable threshold for equilibration can be computed.

Thus, one can discard unequilibrated parts of an DFT-MD trajectory and arrives at a large number of ionic configurations. These are further

processed using real-space distance metrics as outlined in Ref. [**?**]. Here, based on the minimal euclidean distance between any two ions of pairs of ionic configurations within a trajectory, one samples the trajectory until a set of reasonably different ionic configurations has been identified. The process is explained in detail in Ref. [**?**] and has been implemented in the MALA code.

This set of ionic configuration forms the basis for a training data set in the LDOS-based ML-DFT formalism. For each ionic configuration, volumetric data encoding ionic and electronic structure has to be computed. The entirety of metadata, volumetric ionic and electronic structure data per ionic configuration will be referred to as *snapshot* in the following.

To encode the ionic structure of a snapshot on a computational grid, different approaches may be employed that generally follow considerations for computing descriptors of ionic configurations as a whole. Two prominent examples in this regard are the Spectral Neighborhood Analysis Potential (SNAP) and Atomic Cluster Expansion Descriptors (ACE), both of which may be adapted to a computational grid approach. A general overview of descriptors is given in Ref. [**?**].

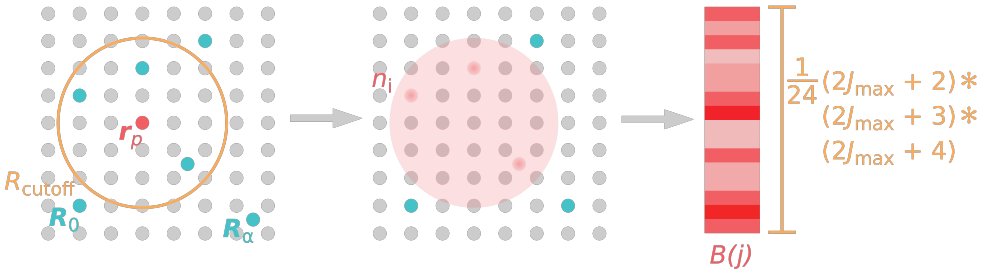

Figure 2: Calculation of descriptors, i.e., representations of local ionic structure on the numerical grid, as given by the bispectrum formalism. Figure adapted from Ref. [**?**].

In its original form, the LDOS-based ML-DFT framework discussed here has been developed using bispectrum descriptors based on the SNAP formalism, as detailed in Ref. [**?**]. Here, one firstly computes an ionic density n_i around each point in space $\boldsymbol{r}_p$, with p indexing this point within the numerical grid, as

$$n_\mathrm{i}(\boldsymbol{r}) = \delta(\boldsymbol{0}) + \sum_{\alpha=1}^{|\boldsymbol{R}_{\alpha,p}|<R_\mathrm{cutoff}^{\eta\alpha}} \left[f_\mathrm{c}\left(\boldsymbol{R}_{\alpha,p}, R_\mathrm{cutoff}^{\eta_\alpha}\right) w_{\eta_\alpha} \delta(\boldsymbol{R}_{\alpha,p}) \right] . \tag{2}$$

Here, $R_{\mathrm{cutoff}}^{\eta_\alpha}$ is a cutoff radius around $\boldsymbol{r}_p$, while w_{η_α} is a dimensionless weight; η, which indexes the chemical species of the ion will be discarded in the following for the sake of brevity.

The precise nature of calculating $n_{\mathrm{i}}(\boldsymbol{r})$ may vary between descriptors [**?**] and, e.g., Gaussian function may be employed instead of δ-distributions. In contrast, the need to further $n_{\mathrm{i}}(\boldsymbol{r})$ both for ensuring computational feasibility and conservation of relevant invariances is more universal. In the context of bispectrum descriptors, vectors $B(j)$ are computed for each $\boldsymbol{r}_p$ by first expressing n_{i} as a sum of hyperspherical harmonic functions $\mathbf{U}_j$ with degree j and expansion coefficients $\mathbf{u}_j$ as

$$n_{\mathrm{i}}(\boldsymbol{r}) = \sum_{j=0,\frac{1}{2},\ldots}^{\infty} \mathbf{u}_j \cdot \mathbf{U}_j(\theta_0, \theta, \phi) \ . \tag{3}$$

Thereafter, one computes elements of bispectrum vectors B through scalar triple products of expansion coefficients. The resulting bispectrum descriptors are real valued, rotationally invariant and of dimensionality

$$B_{\mathrm{max}} = \frac{1}{24}(2J_{\mathrm{max}} + 2)(2J_{\mathrm{max}} + 3)(2J_{\mathrm{max}} + 4) \ , \tag{4}$$

where J_{max} denotes the order of this basis set expansion. This process is further visualized in Fig. **??**. Naturally, the fidelity of this encoding rests on identifying suitable choices for J_{max} and R_{cutoff}. They constitute hyperparameters of the ML-DFT approach, which may be identified based on traditional hyperparameter optimization methods or more efficiently by exploiting the underlying physical nature of their derivation [**?**].

With the descriptors in place, one finally samples the LDOS. In contrast to Eq. **??**, which is formally exact, practical calculation of the LDOS within a plane-wave DFT simulation code takes the form of

$$d(\epsilon, \boldsymbol{r}) = \frac{1}{N_k} \sum_{k=1}^{N_k} \sum_{j=1}^{N'_{\mathrm{e}}} |\psi_{j,\boldsymbol{k}_k}(\boldsymbol{r})|^2 \tilde{\delta}(\epsilon - \epsilon_j) \ , \tag{5}$$

where a summation over a set of $\boldsymbol{k}$-points in Fourier space is performed. The δ-distribution is approximated via a Gaussian function as

$$\tilde{\delta}(x) = \frac{\exp\left(-\frac{x^2}{w_d^2}\right)}{\sqrt{\pi w_d^2}} \ , \tag{6}$$

with width w_d.

Who? Attila, Lenz, Normand, Aidan

- Explain descriptors (somewhat briefly, since we do not have custom descriptors but rather use a variation of SNAP)
- Explain challenges when sampling LDOS, *Figure:(L)DOS sampling plots we had in the original PRB; LF redid those for beryllium, we could show two elements here*
- Reference sampling strategies for trajectories, cite OF-DFT paper
- @SNL: Should we mention ACE on the grid here?

3.3. Models and hyperparameter optimization

Who? Lenz, Bartek

- Mention that we use FF-NN at the moment - @Bartek/@Attila: Should we mention Graph NNs?
- Broad overview of implemented techniques
- Focus on training-free strategies
- @Bartek: Discuss the mutual information ACSD alternative
- *Figures: Hyperopt comparison from paper, mutual information figure*

3.4. Data management

Who? Attila, Lenz

- Explain lazy-loading and inter-snapshot shuffling
- *Figures: data management figure from Lenz' dissertation*

3.5. Observable calculation

Who? Normand, James, Lenz

- Describe current progress on forces
- Maybe mention DOS integration? Or is it enough to link the PRB paper?

3.6. Parallelization and hardware acceleration

Who? Lenz, Jon, Drew

- Explain general parallelization strategy (*Figure: parallelization overview from Lenz' dissertation*)
- Mention specific implementation of structure factor, which requires Gaussian descriptors
- From this, introduce GPU acceleration
- Specifically mention Kokkos usage and multi-GPU inference

4. Computational workflow

Who? Attila, Lenz

- Provide a verbatim example of using MALA by going through a typical workflow
- Example system: Aluminum or beryllium at room temperature (We could also go for a different metal to broaden the appeal? Lithium maybe? Also, wasn't there a suggestion of adding in silicon results?)
- Also show typical outputs of the general workflow (code snippets may be a good idea)

4.1. Data generation

- We could show some trajectory analysis, overall smaller section

4.2. Model training

- Maybe also mention hyperparameter optimization
- Provide loss curves with multiple metrics (Bartek has opened a PR for this)
- Mention multi-GPU training, showcase full results later

4.3. Inference and application

- Showcase how comparatively little code is needed to access several observables
- Mention ASE interface, hint at LAMMPS/CP2K interfaces?

5. Numerical benchmarks

5.1. Accuracy and transferability

Who? Attila

- Overview of size transfer, temperature transfer and phase boundary results
- *Figures: Use updated results from Lenz' dissertation (especially for phase boundaries!)*
- We could also show robustness w.r.t. initialization, Lenz' dissertation has numbers for that

5.2. Computational scaling

Who? Attila, Lenz

- Discuss strong and weak inference scaling, i.e., Figs. 3 and 5
- Discuss scaling up to ultra-large length-scales, see Fig. 6 (Should I add the missing data points for GPUs?)
- Discuss scaling of individual inference portions, see Figs. 4 and 7

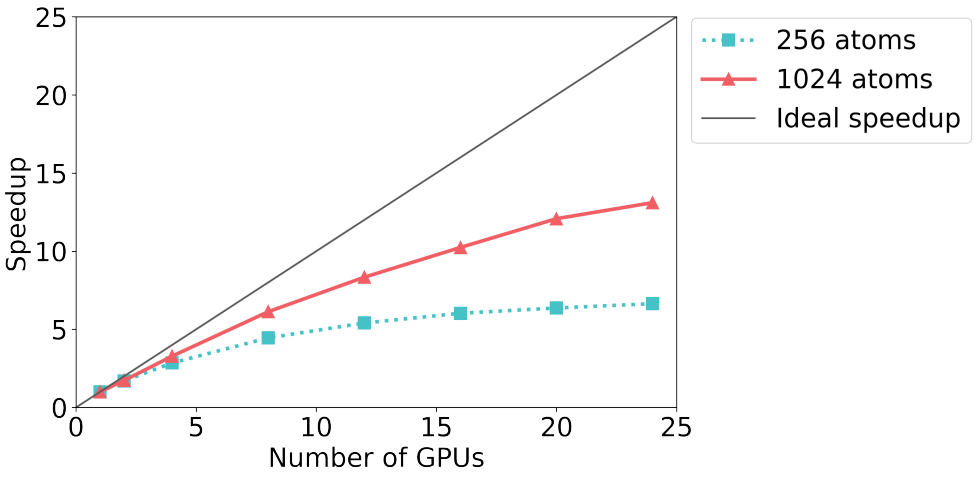

Figure 3: Strong scaling for MALA inference, using the room temperature aluminum model from the PRB paper.

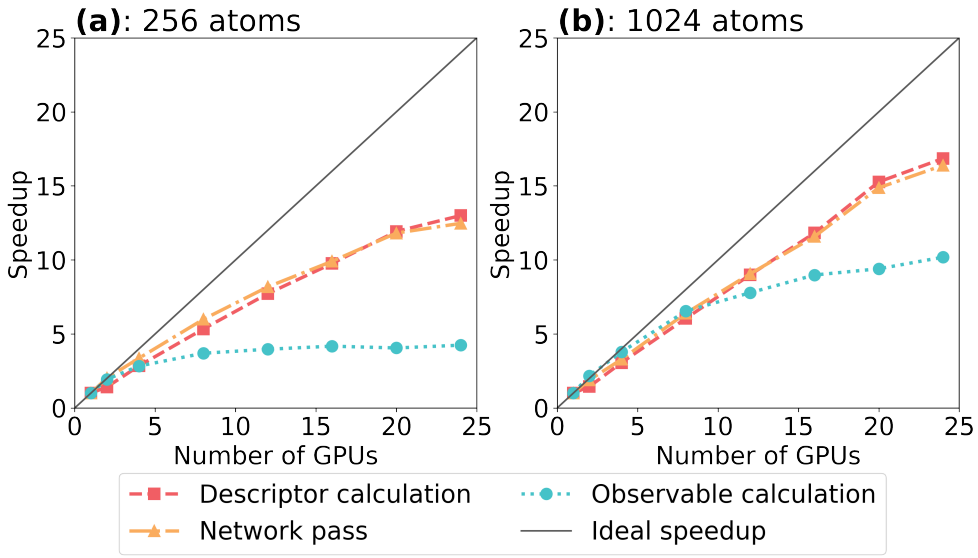

Figure 4: Strong scaling for MALA inference, using the room temperature aluminum model from the PRB paper, split up by calculation portions. Observable calculation means total energy calculation.

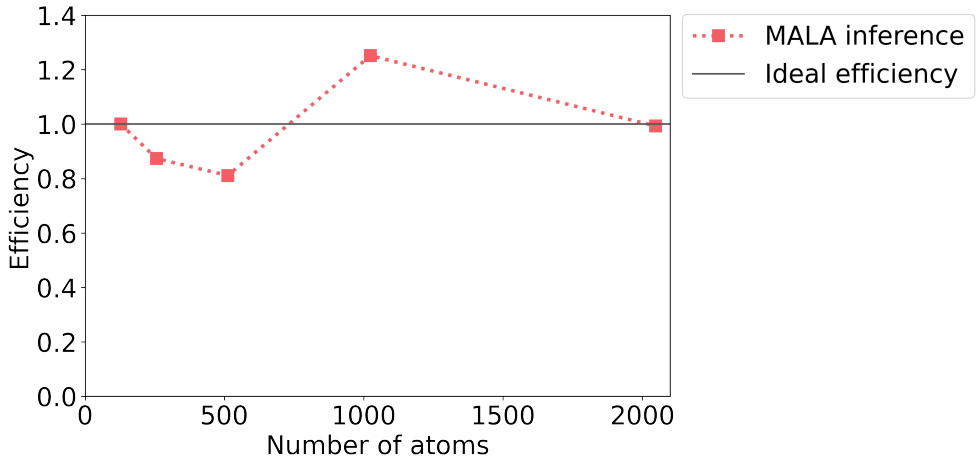

Figure 5: Weak for MALA inference, using the room temperature beryllium model from the size transfer paper. It is a bit spurious that for 512 we have a super-optimal efficiency, but may be explained due to the small overall inference time (roughly 12 seconds). I can redo these calculations if necessary.

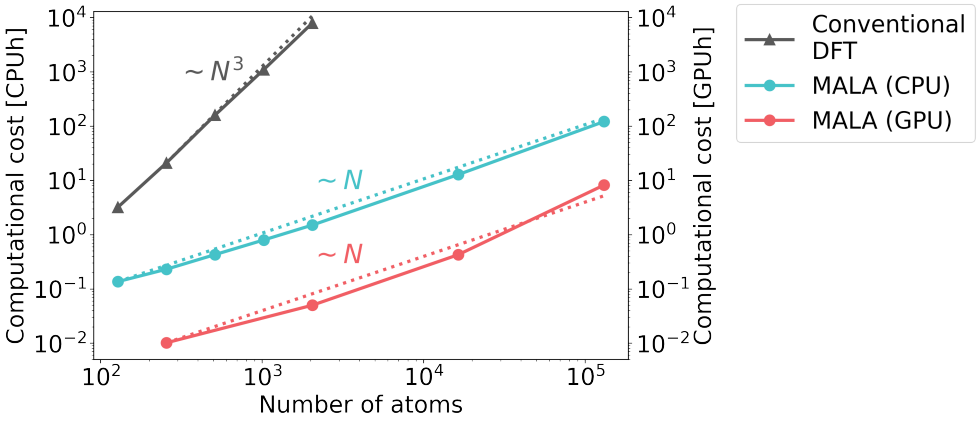

Figure 6: Updated scaling plot from size transfer paper including GPU inference up to ultra-large length scales.

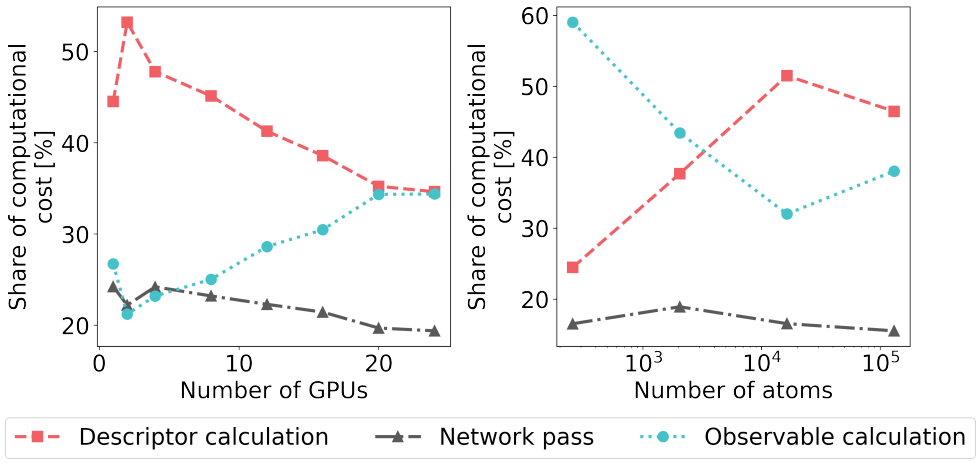

Figure 7: Shares of calculation time for different aspects of MALA inference and scaling behavior w.r.t. number of GPUs and number of atoms. Shows some interesting trends I think.

6. Conclusion

Who? Attila, Lenz

- In this work we have provided the technical details of the Materials Learning Algorithms (MALA) package.
- Describe the main features and capabilities of MALA.
- Provide an outlook on future development of the code
- Restate the application domain of MALA and put it into the context of current research.